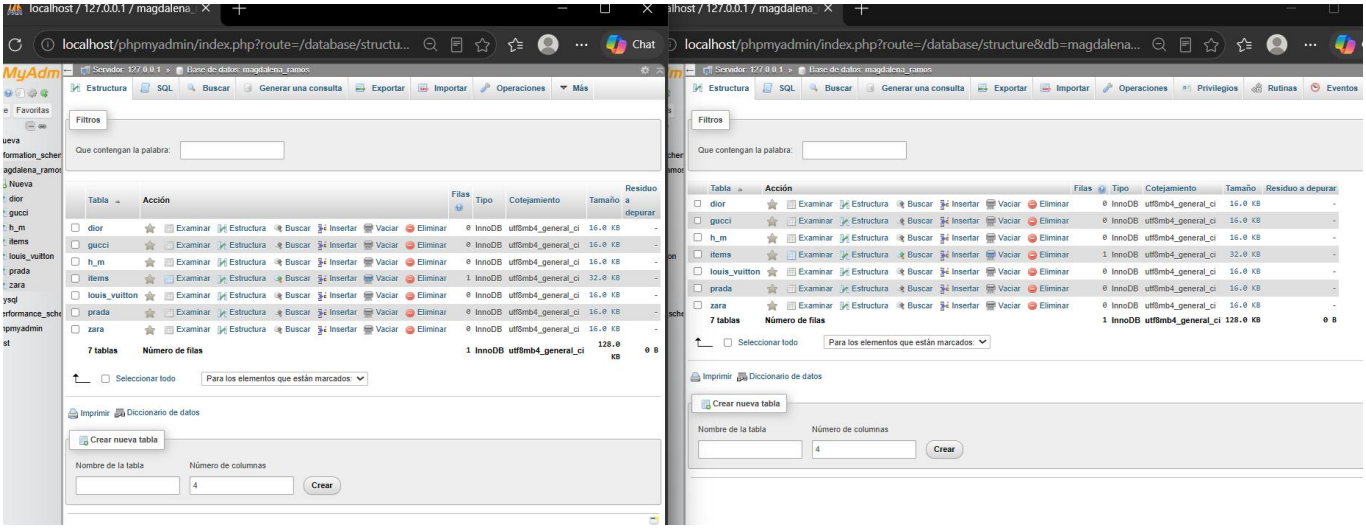
SESIÓN 03: TRÁFICO MASIVO Y CONCURRENCIA

Base de datos · Evento de lanzamiento · MySQL / phpMyAdmin

Preparación

Abre dos pestañas de phpMyAdmin y, en ambas, selecciona la misma base . Así simularás dos clientes comprando a la vez el mismo artículo exclusivo.



Actividad 1 · Columnas para el evento (sin detener el servicio)

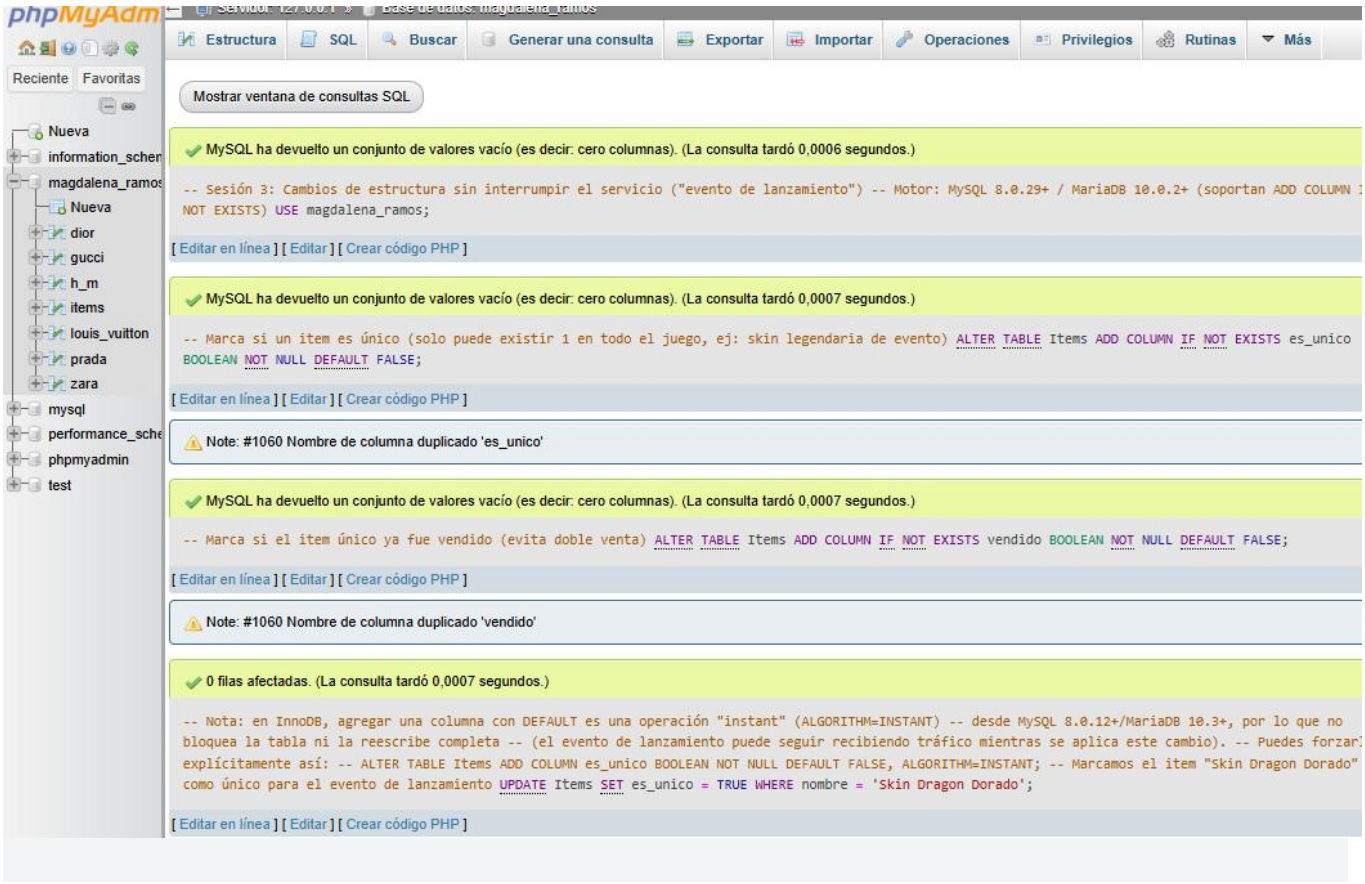
Se agregan a ropa las columnas es_unico (prenda exclusiva) y vendido (candado anti doble venta). En InnoDB reciente, un ADD COLUMN con DEFAULT suele ser INSTANT: no reescribe la tabla ni corta el tráfico.

```
USE magdalena_ramos;

ALTER TABLE ropa
  ADD COLUMN IF NOT EXISTS es_unico BOOLEAN NOT NULL DEFAULT FALSE;

ALTER TABLE ropa
  ADD COLUMN IF NOT EXISTS vendido BOOLEAN NOT NULL DEFAULT FALSE;

UPDATE ropa SET es_unico = TRUE WHERE nombre = 'Vestido Elegante Noche';
```



Qué hace: IF NOT EXISTS evita error si el script se ejecuta dos veces. El UPDATE marca el Vestido Elegante Noche como pieza única del evento.

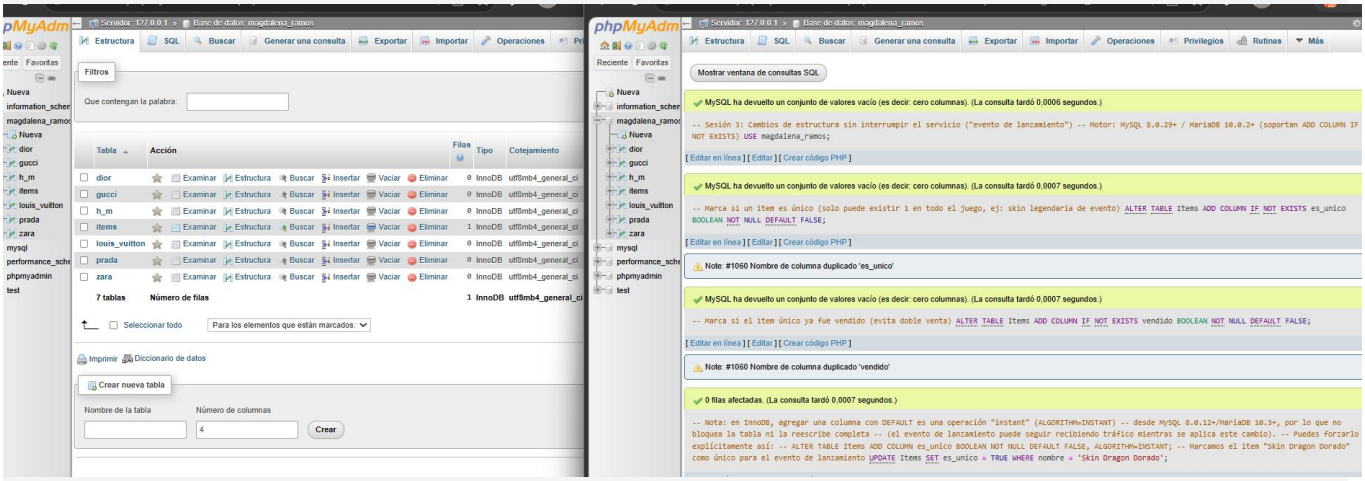
Actividad 2 · Procedimiento que evita la doble venta

Sin control, dos clics simultáneos pueden leer «disponible» y entregar dos veces el mismo ítem. La solución combina:

- Transacción: verificar, cobrar y entregar en un solo bloque (todo o nada).
- SELECT ... FOR UPDATE: bloquea la fila; la segunda sesión espera.
- ON DUPLICATE KEY UPDATE: si ya tenía la prenda, suma cantidad.

```
USE magdalena_ramos;
DROP PROCEDURE IF EXISTS comprar_ropa_unico;
DELIMITER //
CREATE PROCEDURE comprar_ropa_unico(
    IN p_cliente_id INT, IN p_ropa_id INT, OUT p_resultado VARCHAR(255))
BEGIN
  DECLARE v_vendido BOOLEAN; DECLARE v_precio INT; DECLARE v_monedas INT;
  START TRANSACTION;
  SELECT vendido, precio INTO v_vendido, v_precio
  FROM ropa WHERE ropa_id = p_ropa_id AND es_unico = TRUE FOR UPDATE;
  IF v_vendido IS NULL THEN
    SET p_resultado = 'RECHAZADO: El ítem no existe o no es único.'; ROLLBACK;
  ELSEIF v_vendido THEN
    SET p_resultado = 'RECHAZADO: El ítem único ya fue vendido.'; ROLLBACK;
  ELSE
    SELECT monedas INTO v_monedas FROM clientes WHERE cliente_id = p_cliente_id;
    IF v_monedas IS NULL OR v_monedas < v_precio THEN
      SET p_resultado = 'RECHAZADO: Monedas insuficientes.'; ROLLBACK;
    ELSE
      UPDATE ropa SET vendido = TRUE WHERE ropa_id = p_ropa_id;
      UPDATE clientes SET monedas = monedas - v_precio WHERE cliente_id = p_cliente_id;
```

```
      INSERT INTO Inventario_ropa (cliente_id, ropa_id, cantidad)
        VALUES (p_cliente_id, p_ropa_id, 1)
        ON DUPLICATE KEY UPDATE cantidad = cantidad + 1;
      SET p_resultado = CONCAT('ÉXITO: Ítem único vendido al cliente ', p_cliente_id);
      COMMIT;
    END IF;
  END IF;
END //
DELIMITER ;
```



Actividad 3 · Simular el lanzamiento (dos pestañas)

Cliente 2 y cliente 5 intentan comprar la misma prenda única (ropa_id = 1) casi al mismo tiempo.

Pestaña A (cliente 2)

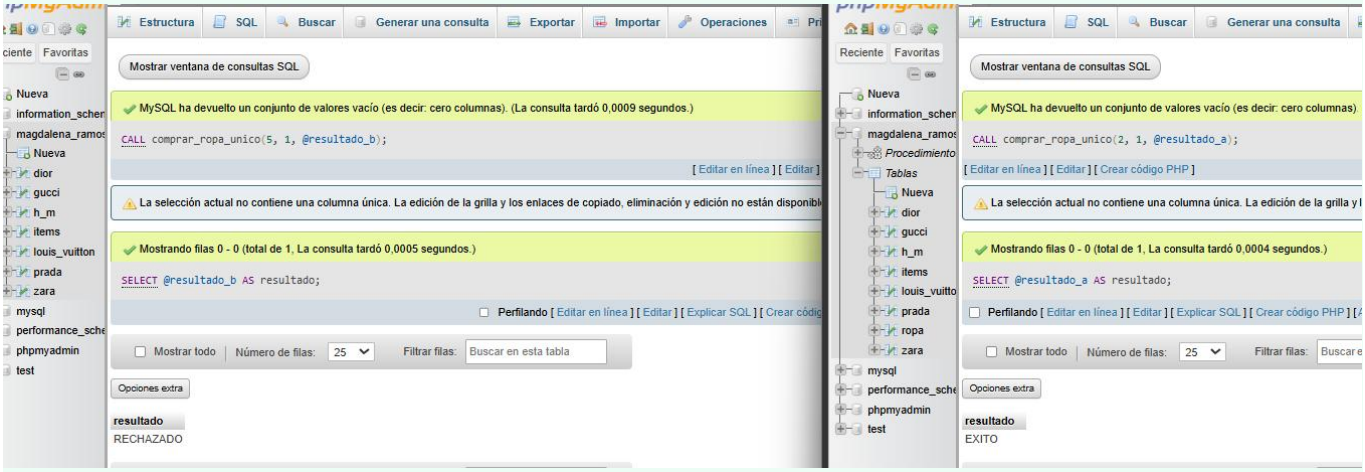
```
CALL comprar_ropa_unico(2, 1, @resultado_a);
SELECT @resultado_a AS resultado;
```

Pestaña B (cliente 5)

```
CALL comprar_ropa_unico(5, 1, @resultado_b);
SELECT @resultado_b AS resultado;
```

Pulsa Continuar en A y de inmediato en B.

Resultado esperado: una pestaña muestra ÉXITO y la otra RECHAZADO. Eso confirma que FOR UPDATE funcionó.



Conclusión de la sesión

Las transacciones y el bloqueo de fila convierten una carrera peligrosa en una cola ordenada: solo una compra gana. ALTER TABLE INSTANT permite preparar el evento sin apagar la tienda.